

You've seen that something is reduced in size each time a recursive method calls itself and built back up again each time the method returns. In mergesort, the range of cells is divided in half each time this method calls itself, and each time it returns it merges two smaller ranges into a larger one.

As the mergesort algorithm returns from processing two arrays of one element each, it merges them into a sorted array of two elements. Each pair of resulting two-element arrays is then merged into a four-element array. This process continues with larger and larger arrays until the entire array is sorted. This sort is easiest to see when the original array size is a power of 2, as shown in [Figure 6-16](#). The input, unsorted array is shown at the left, with the first cell (index 0) at the top. Time elapses moving toward the right. The recursive calls split the eight-element array into halves of four, then quarters of two, and finally into the base case call on a single element.

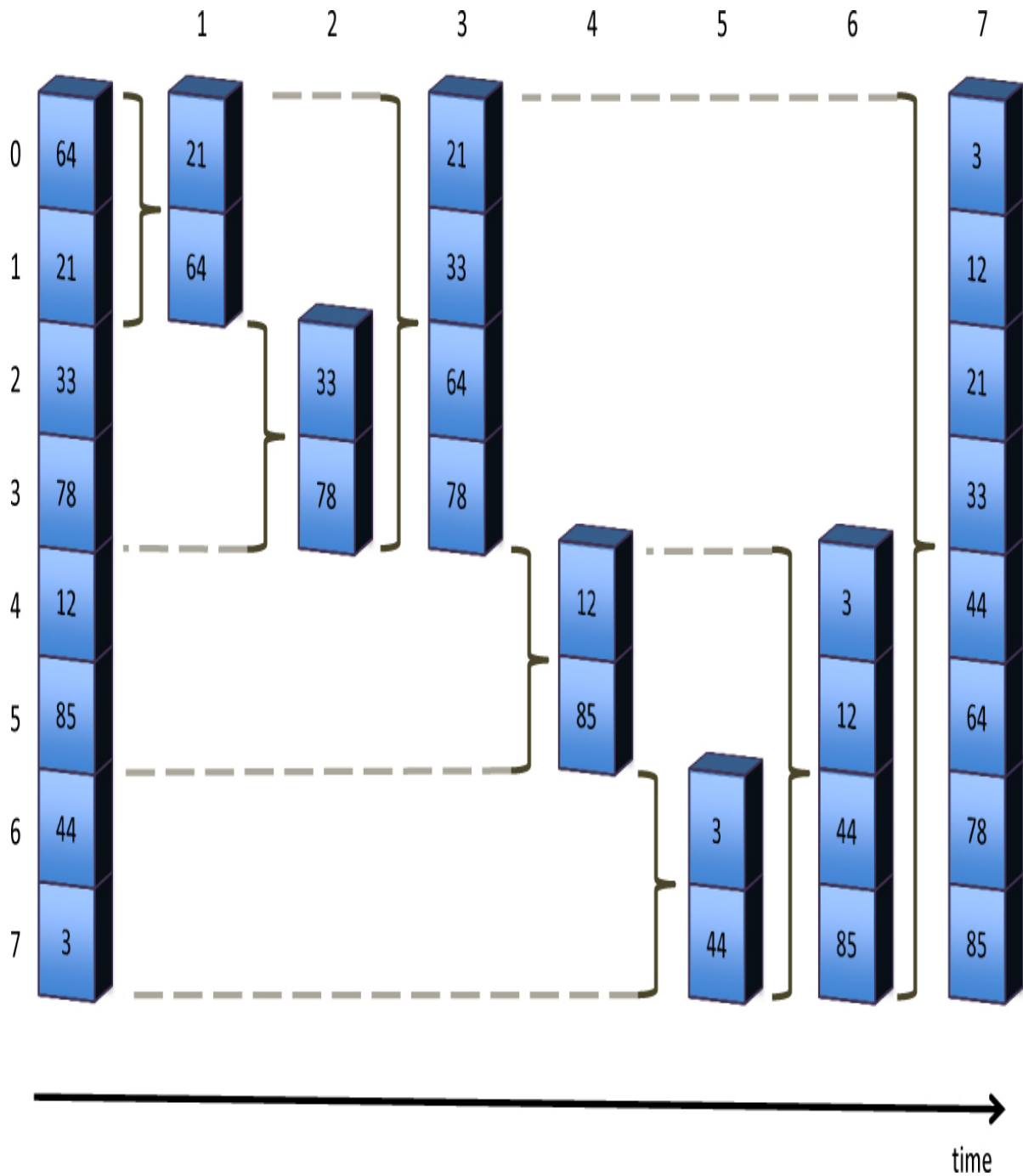


Figure 6-16 *Mergesorting an eight-element array*

Each recursive call works on a range of the array cells. You can use Python slice notation to indicate the ranges being worked in each call. The full array is [0:8], and the first quarter is [0:2]. Note the second index of the slice is one past the last index of the range. Mathematicians might prefer the notation [0,8)

for the range of integers 0, 1, 2, 3, 4, 5, 6, and 7, but Python uses `[0:8]`. We use both in different parts of this book.

Let's assume you have a recursive implementation called `mergesort()` that works on a range in the array. The first call to it is with the full array, `[0:8]`. Because it's not the base case of a single cell, it makes a recursive call to `mergesort()` the first half of the array, `[0:4]`. That too is not a base case, so it makes a recursive call to `mergesort()` the first quarter, `[0:2]`, and finally the first eighth, `[0:1]`. That is a single cell range, which is already sorted, so that call returns immediately. The same is true of the second eighth, the range `[1:2]`.

The second column (numbered 1) in [Figure 6-16](#) shows the merge step where the first two eighths are merged into a sorted quarter, range `[0:2]`, moving 21 to precede 64. This quarter range is passed back to the recursive call to `mergesort()` on the first half, `[0:4]`. That call now goes on to `mergesort()` the second quarter, `[2:4]`. The third column (numbered 2) in [Figure 6-16](#) shows the merge step on the eighths that make up that quarter. There is no change in the values because they were already in sorted order.

The fourth column in [Figure 6-16](#) shows the merging of the first two sorted quarters. The lowest values are chosen first in zipper-like order to make the sorted half. The same process then repeats to dive into the second half of the array, breaking it into quarters, then eighths, and merging the sorted results. In the far-right column of [Figure 6-16](#), the full array is sorted by merging the two sorted halves.

When the array size is not a power of 2, arrays of different sizes must be merged. For example, [Figure 6-17](#) shows the situation when the array size is 10. Here an array of size 2 must be merged with an array of size 1 to form an array of size 3.

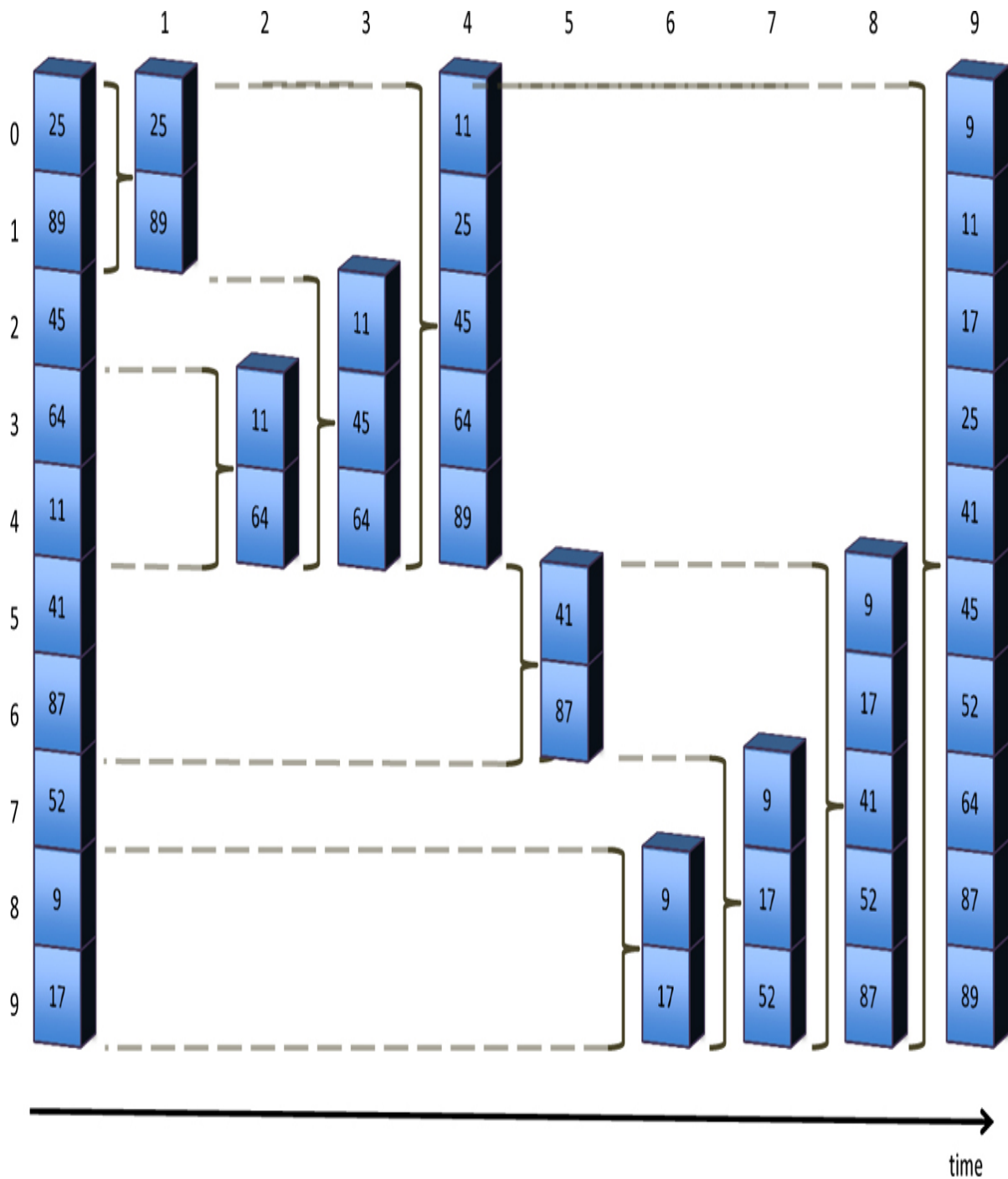


Figure 6-17 Mergesort when the array size not a power of 2

As `mergesort()` makes its recursive calls to subdivide the range, it first divides the 10 cells into two ranges of 5 cells each. Those are then subdivided into arrays of size 2 and 3. The two-cell arrays are processed as above, but in the case of the three-cell array, it must be split into unequal subranges. For example, the first half, [0:5], is split into subranges [0:2] and [2:5]. The [2:5]

subrange is split into [2:3] and [3:5]. The [2:3] subrange has only one cell, so it is handled as the base case where no sorting needs to be done. The fourth column of [Figure 6-17](#) shows the merge of the one- and two-cell arrays into the three-cell array.

You may wonder how much memory is required to hold all these smaller arrays during the mergesort. To merge two arrays of size M and N together, the algorithm needs another M+N size array to hold the result. Reviewing the basic algorithm:

1. Sort the first half of the array
2. Sort the second half of the array
3. Merge the two sorted halves together

only the third step requires temporary storage. The first two steps can be done by putting the result back in the input array cells. So, if you expand step 3 to be

- a. Merge the two sorted halves of the input array into a temporary array
- b. Copy the temporary array back into the input array

you will need a full copy of the input array, but nothing more, for all the recursive steps. The implementation shown in [Listing 6-4](#) uses a work array to hold the temporary results.

Listing 6-4 *The Mergesort.py Module*

```
def identity(x): return x          # Identity function

from Array import *

class Mergesort(object):          # An object to mergesort Arrays
    def __init__(self,            # Constructor takes the unordered
        unordered,                # array and orders its items by using
        key=identity):            # mergesort on their keys
        self.__arr = unordered    # Array starts unordered
        self.__key = key          # Key func. returns sort key of item
        n = len(unordered)        # Get number of items
        self.__work = Array(n)    # A work array of the same length
        for i in range(n):        # is needed to rearrange the items
            self.__work.insert(None) # Work array is filled with None
        self.mergesort(0, n)      # Call recursive sort on full array
```

```
def mergesort(self, lo, hi): # Perform mergesort on subrange
    if lo + 1 >= hi:        # If subrange has 1 or fewer items,
        return              # then it is already sorted
    mid = (lo + hi) // 2    # Otherwise, find middle index
    self.mergesort(lo, mid) # Sort the lower half of subrange,
    self.mergesort(mid, hi) # Sort the upper half of subrange,
    self.merge(lo, mid, hi) # Merge the 2 sorted halves
```

The `Mergesort` class is defined with a single purpose—to sort array objects. This example reuses the `Array` class defined in [Chapter 2](#) for the input and output array. Each `Mergesort` object will be constructed from a single `Array` object. The constructor method takes the unsorted array and a `key` function to get the sort key from each item in the array. The default key function is the identity function that returns the item itself, which can be used to sort an array of primitive types like integers or characters.

The constructor adds private attributes for the `__arr` array and the sort `__key` function. It allocates a `__work` array that is the same size as the input array. All of the recursive calls will be able to use the same work array because they each copy their result back into the input array. The work array is filled with the `None` value to ensure all the cells are allocated. Finally, the constructor calls `mergesort()` on the entire array range, `[0:n]`. There is no return statement because the results will be stored in the input/output array by `mergesort()`.

The recursive `mergesort()` method is defined to work on subranges of the array between a `lo` and `hi` index (`hi` is one past the highest index). As usual, the first step is to check for the base case, a 1-cell array subrange. It also looks for a 0-cell or empty range just in case the input array was empty. In either of those cases, the input subrange is already sorted, and the method can return without any further action.

For subranges larger than 1, `mergesort()` computes a midpoint index between `lo` and `hi` to divide the subrange into two approximately equal halves, `[lo:mid]` and `[mid:hi]`. Recursive calls sort these subranges. Finally, the two sorted subranges are merged together in the call to `merge()`.

Merging Subranges

The `mergesort()` method calls `merge()` to merge the items from the sorted subranges, making use of the work array as shown in [Listing 6-5](#).

Listing 6-5 *The Mergesort.py Module's merge() Method*

```
def merge(self, lo, mid, hi): # Merge 2 sorted subranges of input
    n = 0                    # into work array which starts empty
    idxLo = lo               # Use indices into lo and hi
    idxHi = mid              # subranges to track next items
    while (idxLo < mid and    # Loop until one of the subranges
           idxHi < hi):      # is empty
        itemLo = self.__arr.get(idxLo) # Get next items from the
        itemHi = self.__arr.get(idxHi) # two subranges
        if (self.__key(itemLo) <=      # Compare keys of those items
            self.__key(itemHi)):
            self.__work.set(n, itemLo) # Lo subrange is first so
            idxLo += 1                 # copy item and advance to next
        else:
            self.__work.set(n, itemHi) # Hi subrange is first so
            idxHi += 1                 # copy item and advance to next
        n += 1                        # One more item now in work array

    while idxLo < mid:         # Loop to copy remaining lo
        self.__work.set(      # subrange items to work array
            n, self.__arr.get(idxLo))
        idxLo += 1
        n += 1

    while n > 0:              # Copy sorted work array contents
        n -= 1                # back to input/output array in
        self.__arr.set(       # reverse order
            lo + n, self.__work.get(n))
```

The `merge()` method works with two, adjacent subranges of the array stored in the `__arr` attribute of the object. It steps through the subranges, copying the lowest value in each one to the `__work` array that was created by the constructor. It uses `n` to count the number values that have been copied and `idxLo` and `idxHi` to index the two subranges. The two indices start at the lowest index of their respective ranges and work up.

The first `while` loop in `merge()` handles the case when there are values to compare from both subranges. It checks that both indices are still within their valid ranges and gets the items at those indices, `itemLo` and `itemHi`. It compares the sort keys by applying the sort `key` function to each one. Whichever item has the lower sort key is copied to the work array. The count

of copied items is advanced along with the index for the corresponding subrange.

After one of the subranges becomes empty, no more comparisons are needed, just copying values to their proper positions. The `merge()` method copies any remaining items in the low subrange in the second `while` loop. Finally, the entire `__work` array can be copied back into the subrange of the `__arr` array starting at `lo`. The last `while` loop decrements `n` back to 0 instead of using another index variable for this copying.

Does something look strange in `merge()`? In particular, what about copying any remaining items in the high subrange to the `__work` array? In the previous examples and in [Figure 6-15](#), it shows copying all the items to the work array. Why is that missing in `merge()`?

Because the input array is also the output array, copying the highest part of a range into the work array and then back again accomplishes nothing. When the first `while` loop is done, one of the two ranges has been fully copied to the work array. If it's the lower range, then the second `while` loop has nothing to do. That leaves some or all of the higher range in the input array, `idxHi < hi`, but all those items must have higher key values than what has already been copied. You could copy them to the work array and increment `n`, but they would just be copied back to the same cells by the third `while` loop. Thus, the method skips the unneeded copying and leaves them unchanged.

Testing the Code

The `MergesortClient()` shown in [Listing 6-6](#) tests the basic operation of `Mergesort()`.

Listing 6-6 *The MergesortClient() Module*

```
from Mergesort import *
from Array import *

values = [19, 49, 70, 72, 43, 80, 95, 46, 19, 18, 45, 6, 56, 85]
array = Array(len(values))
for value in values:
    array.insert(value)

print('Initial array contains', len(array), 'items')
```

```
array.traverse()
```

```
Mergesort(array)
```

```
print('After applying Mergesort, array contains', len(array), 'items')  
array.traverse()
```

Because the `Array` class introduced in [Chapter 2](#) doesn't have a `__str__()` method, the test client uses the `traverse()` method to show its contents.

Running the test shows

```
$ python3 MergesortClient.py  
Initial array contains 14 items  
19  
49  
70  
72  
43  
80  
95  
46  
19  
18  
45  
6  
56  
85  
After applying Mergesort, array contains 14 items  
6  
18  
19  
19  
43  
45  
46  
49  
56  
70  
72  
80  
85  
95
```

The Mergesort Visualization Tool

This sorting process is easier to appreciate when you see it happening before your very eyes. Start up the Mergesort Visualization tool. The tool starts with a small array of random numbers. You can create a new array of a particular size using the now-familiar New button and fill it with more random integers using the Random Fill button. You start the mergesort process by selecting the Mergesort button. The first thing it does is create the work array near the bottom to hold the merged ranges. Then the mergesort algorithm starts dividing the array into halves, quarters, and so on. The range of cells that it is working on is moved down for each recursive level. When it reaches a merge step, it copies items into the work array. [Figure 6-18](#) shows an example as it merges a two-cell range of the array.



Figure 6-18 *The Mergesort Visualization tool*

The range of cells is split at the `mid` index that's calculated by `mergesort()`. As it descends a level, the range from `lo` to `mid` is moved a little to the left, and the range from `mid` to `hi` is moved a little to the right. The `lo`, `mid`, and `hi` indices change at each level, and the visualization shows only the `mid` index to reduce clutter. It leaves the `mid` index of higher levels in a fainter color and restores the index when the recursion returns to that level.

The `idxLo` and `idxHi` indices show which cells the `merge()` method is examining. The lower keyed item is copied to the work array. After one of the ranges is fully copied, it copies anything remaining in the low range and then copies all `n` items back into the input array cells. The cells move back up as the recursive call ends.

As the algorithm runs, watch the code window. The recursive calls to `mergesort()` stack up, each with different ranges to sort. At the end of each

call to `mergesort()`, it pushes the call to `merge()` on the stack to merge the two subranges. You can step through or pause the animation to look at the details of the call stack to see what the different loops do.

Efficiency of the mergesort

Although the bubble, insertion, and selection sorts described in [Chapter 3](#) take $O(N^2)$ time, the mergesort is $O(N \times \log N)$. The graph at the end of [Chapter 2](#) shows how much faster this is. For example, if N (the number of items to be sorted) is 10,000, then N^2 is 100,000,000, while $N \times \log N$ is only 40,000 (using $\log_{10}$). If sorting this many items required 40 seconds with the mergesort, it would take almost 28 hours for the insertion sort.

How do you know the mergesort takes $O(N \times \log N)$ time? That's not easy to see just by looking at the algorithm or the implementation, and it's one of the most challenging and interesting parts of computer science. To figure it out, you can count the number of times a data item must be copied and the number of times it must be compared with another data item during the course of the algorithm. We assume that copying and comparing are the most time-consuming operations and that the recursive calls and returns don't add much overhead.

Number of Copies

Consider [Figure 6-16](#), which showed the sorting of an eight-cell array. Each cell to the right of the left-hand column represents an element copied from the input array into the work array. More accurately, they are cells that *could have been copied* into the work array because, as we discussed previously, the items left in the high range didn't need to be copied. Let's assume the worst case, however, and count all the cells as being copied. (We'll handle the copying back to the input/output array shortly.)

Adding up all the cells in [Figure 6-16](#) (the seven columns to the right) shows there are 24 copies necessary to sort 8 items. $\log_2(8)$ is 3, so $8 \times \log_2(8)$ equals 24. This shows that, for the case of 8 items, the number of copies is proportional to $N \times \log_2 N$.

Another way to look at this calculation is that to sort 8 items requires 3 *levels*, each of which involves 8 copies. A level means all copies go into the same size

subrange of the array (or subarray). In the smallest subrange level, there are four 2-cell subarrays; in the next larger level, there are two 4-cell subarrays; and in the third level, there is one 8-cell subarray. Each level has 8 elements, so again there are 3×8 , or 24, copies. The levels correspond to the recursive calls to `mergesort()` that divide the range in half.

You can use [Figure 6-16](#) to figure out the number of copies needed for a four-cell array by only considering the top four cells of the array. To sort those cells, eight copies are needed. Checking the formula for that size shows $4 \times \log_2(4) = 4 \times 2 = 8$. Going one level lower, two copies are necessary for two items. Similar calculations provide the number of copies necessary for larger arrays. [Table 6-4](#) summarizes this information.

Table 6-4 *Number of Operations When N Is a Power of 2*

N	$\log_2 N$	Number of Copies into Work Array ($N \times \log_2 N$)	Total Copies	Comparisons Max (Min)
2	1	2	4	1 (1)
4	2	8	16	5 (4)
8	3	24	48	17 (12)
16	4	64	128	49 (32)
32	5	160	320	129 (80)
64	6	384	768	321 (192)
128	7	896	1,792	769 (448)

The items are not only copied into the work array but are also copied back into the original array. In the worst case, this process doubles the number of copies, as shown in the Total Copies column. The final column of [Table 6-4](#) shows comparisons, which we cover in a moment.

It's harder to calculate the number of copies and comparisons when N is not a multiple of 2, but these numbers fall between those that are a power of 2. For 10 items, there are 34 cells in [Figure 6-17](#), which means there are at most $34 \times 2 = 68$ total copy operations. That counts lies between the counts for $N = 8$ and $N = 16$, that is, between 48 and 128.

Number of Comparisons

In the mergesort algorithm, the number of comparisons is always somewhat less than the number of copies depending on the amount of sorting that's needed. How much less? Assuming the number of items is a power of 2, for each individual merging operation, the maximum number of comparisons is always one fewer than the number of items being merged, and the minimum is half the number of items being merged. That minimum is called the **best case**, and it happens when the input array is already in sorted order. The merge copies all the items from the lower half of the array while doing the comparisons. The second loop does nothing because all the lower items are in the work array, and then the final loop copies the lower half back to the input array. The maximum number is the **worst case** and happens when the item's sort keys interleave. The items are copied zipper-like into the work array, and only the last item requires no comparisons. In the example of [Figure 6-15](#), eight comparisons were needed before the last two items could be merged into the result.

Those cases tell us the range of possible comparisons. To get a specific count for a specific input array, you have to add up all the comparisons at all the levels. Referring to [Figure 6-16](#), you can see that seven merge operations are required to sort eight items. The number of items being merged and the resulting number of comparisons are detailed in [Table 6-5](#).

Table 6-5 *Comparisons Involved in Sorting Eight Items*

Step/Column Number	1	2	3	4	5	6	7	Totals
Number of items being merged (N)	2	2	4	2	2	4	8	24
Maximum comparisons (N-1)	1	1	3	1	1	3	7	17
Minimum comparisons (N/2)	1	1	2	1	1	2	4	12
Example in Figure 6-16	1	1	3	1	1	3	7	17

For each merge, the maximum number of comparisons is one fewer than the number of items. Adding these figures for all the merges gives a total of 17. The minimum number of comparisons is always half the number of items being merged, and adding these figures for all the merges results in 12 comparisons.

Similar arithmetic results in the values shown in the Comparisons column of [Table 6-4](#).

The actual number of comparisons to sort a specific array depends on how the input items are arranged, but it will be somewhere between the minimum and maximum values. Because both the best and worst cases depend on N , the average number does too. That means that at every recursion level, there will be $O(N)$ comparisons. Because there are $O(\log N)$ levels of recursion, there will be $O(N \times \log N)$ comparisons overall.

Eliminating Recursion

Some algorithms lend themselves to a recursive approach; some don't. As you've seen, the recursive `triangular()` and `factorial()` functions can be implemented more efficiently using simple loops. Various divide-and-conquer algorithms, however, work very well as recursive routines.

Often an algorithm is easy to conceptualize as a recursive method, but in practice the recursive approach proves to be inefficient. In such cases, it's useful to transform the recursive approach into a nonrecursive approach. Such transformations often make use of a stack.

Recursion and Stacks

There is a close relationship between recursion and stacks. In fact, most compilers implement recursion by using stacks. As we noted, when a method is called, the compiler pushes the arguments to the method and the return address (where control will go when the method returns) on the stack, and then transfers control to the method. When the method returns, it pops these values off the stack. The arguments disappear, and control returns to the return address.

Simulating a Recursive Function: Triangular

In this section we demonstrate how any recursive solution can be transformed into a stack-based solution. Remember the recursive `triangular()` function from the first section in this chapter? Here it is again renamed

```
triangular_recursive():
```

```
def triangular_recursive(nth): # Get the nth triangular number
    if nth < 1: return 0 # For anything less than 1, it's 0
    return (nth +        # Otherwise add this column to the preceding
            triangular_recursive(nth - 1)) # triangular number
```

We use this function to show the basic idea, even though we know that we could really transform this function into one that doesn't require recursion or looping; for example,

```
def triangular(nth): # Get the nth triangular number
    return 0 if nth < 1 else nth * (nth + 1) / 2
```

If we didn't know about that closed form solution, we could use a stack to model the recursive calls. Let's start with a stack that has one item on it that defines the problem to be solved. Then we can iterate until the stack is either empty or has one item with a solution to return. Here's the basic idea:

```
def triangular_via_stack(nth): # Get the nth triangular number using
    todo = LinkStack()        # a stack of problem descriptions
    todo.push([nth, None])    # Description: nth and recursive result
    while not todo.isEmpty(): # Loop until no more problems to solve
        top = todo.peek()    # Look at topmost problem
        if top[1] is None:   # If recursive result is not solved,
            if top[0] < 1:    # check if top is base case
                top[1] = 0    # If so, then no recursion needed
            else:             # Otherwise, solve smaller problem
                todo.push([top[0] - 1, None])
        else:                # Topmost is solved
            top = todo.pop()  # Pop it off the stack
            if todo.isEmpty(): # If it was the last one,
                return top[1] # then return the solution
            else:             # Else add recursive call result to
                caller = todo.peek() # caller's nth which is next on stack
                caller[1] = caller[0] + top[1]
    raise Exception("Stack empty without finding solution")
```

The `todo` variable holds a stack. We choose the `LinkStack` class defined in [Chapter 5, "Linked Lists,"](#) which does not require a maximum size for the stack contents. The problem descriptions placed on the stack are simple lists of two items: a number that is the value of `nth` from the recursive definition and `None`, which is a placeholder for the result being sought. The algorithm will fill in that value eventually.

After initializing the `todo` stack, the loop iterates until the stack is empty. In the loop body, there must be at least one problem to solve, and that is copied to the

`top` variable. The remainder of the loop body performs one of the calls from the recursive version of the function. In the recursive body, it looked first at `nth`, but here we need to look to see if the result value has been filled in for the topmost problem. The reason is that we need to know whether to execute the part of the code before or after the recursive call. It's kind of like the instruction pointer that was pushed onto a call stack telling the interpreter where to resume execution after a recursive call finishes.

When the topmost result is not filled in, which is the way the stack is initialized, the loop body executes the part of the recursive function that comes before the recursive call. That is where it checks whether the problem description is a base case or requires recursion. The base case occurs when the problem is to find the 0th or lower triangular number. That is 0 by definition, so it puts the result in the topmost problem description by setting `top[1]`. If the `top` problem description is for a number 1 or higher, then the recursive case pushes a new, smaller problem description to be solved. That's done by pushing `[top[0] - 1, None]` on the `todo` stack. That's effectively going to perform the recursive call to `triangular_recursive(nth - 1)`. The second component being `None` leaves another placeholder on the stack for the result of that call.

The `else` clause of the outer `if` statement handles when the topmost problem has a solution. That's the point after the recursive call on the smaller problem has returned. This is where the recursive version adds the value `nth` to the recursive result and returns that to its caller. In order to "return" a result, the function needs to store a value in the placeholder created by the caller to hold it. First, it checks if there is a "caller" by seeing whether the stack is empty or not. An empty stack means that it has already popped off the original problem description and can simply return the result that was computed for it—the value in `top[1]`.

When the stack is not empty, `triangular_via_stack()` function "returns" the result by adding the result of the recursive call, `top[1]`, to the value of `nth` in the caller. The caller's problem description is at the top of the stack, so it assigns that pair to the `caller` variable by calling `todo.peek()`. The value of the caller's `nth` variable is stored in `caller[0]`. The result of the caller, `caller[1]`, is updated to be the sum of its `nth`, (stored in `caller[0]`), and the recursive call result, `top[1]`.

The code also has an ending that checks whether the stack is ever completely emptied. That should never happen for triangular numbers, so an exception is

thrown.

To summarize, writing a stack-based version of a recursive function involves

- Creating a stack to hold problem descriptions
- Pushing the original problem description on the stack
- Including all the parameters that would be passed to a recursive call in each problem description
- Including one or more placeholders for the results of the recursive call (if any are returned)
- Looping until the problem description stack is emptied
- Checking the topmost problem description and handling
 - When the problem has not yet been solved, that is, the part before recursive calls where the base cases are handled
 - Adding any recursive calls to the stack
 - When the recursive calls have been solved, combining their results with other problem description parameters, and "returning" results by putting them in placeholders on the stack, if any
- Storing any local variables from the recursive function as items in the problem description. (There aren't any of these in the preceding `triangular_recursive()` function, so you need only the `nth` function parameter and the result placeholder).

This entire process of translating recursive functions into nonrecursive version using stacks can be automated. Some compilers do it for you, but it's important to understand how this process happens. One of the trickiest parts is remembering where all the data gets stored. All the arguments to the recursive function and any local variables must be kept in the problem description on the stack so that all the nested versions of the problem can keep track of their data separately (see [Figures 6-5](#), [6-6](#), [6-7](#), and [6-8](#) for examples). The return values for each recursive call must also be stored there.

Rewriting a Recursive Procedure: mergesort

The process of converting a recursive procedure is almost identical to that of converting a recursive function. Let's convert the `mergesort()` method of [Listing 6-4](#) to look at the differences and similarities.

The `mergesort()` method is the only recursive method of the `Mergesort` class. Because it is part of a class, you can allocate a stack when the object is created to hold the recursive subproblem descriptions. Thus, the class constructor takes care of the first two bullets of the conversion process.

The problem descriptions need to have all the parameters that are passed to the recursive method—namely `lo` and `hi`, which define the subrange of array cells to sort. There is also the `mid` variable that is a local variable of the `mergesort()` method. You can include that in the problem description and put it in the middle so that each description is a list in the form `[lo, mid, hi]`. Each iteration of the loop will examine the topmost problem description on the stack to perform part of the algorithm.

Each loop iteration must handle part of the original routine's processing between recursive calls. [Listing 6-7](#) shows the different parts of the processing using different colors.

Listing 6-7 *Parts of the Recursive `mergesort()` Method*

```
def mergesort(self, lo, hi):    # Perform mergesort on subrange
    if lo + 1 >= hi:           # Initial processing
        return
    mid = (lo + hi) // 2
    self.mergesort(lo, mid)     # Processing after 1st recursive call
    self.mergesort(mid, hi)     # Processing after 2nd recursive call
    self.merge(lo, mid, hi)
```

The first part of the routine includes the processing steps up to the first recursive call. The routine checks the base case and returns if it's found or creates the local variable, `mid`, in the initial part. The second part is the processing done between the recursive calls, which is nothing (but is still important because the subrange has been sorted). The third and last part follows the second recursive call and involves merging the two sorted subranges of the array.

Each iteration of the stack-based processing performs one of these three parts of the routine. It has to decide which part to perform based on the problem

description record. You could put a field in the record called `step` or something similar to keep track of what step to perform, but you can also look at the value of the local variable `mid`. The `mid` variable starts off undefined and then is later filled in with a value that is the average of the `lo` and `hi` variables. If you initially create problem descriptions with `mid` as `None` and then set it to the average or some third value, you can use this field of the problem description to determine what step to perform. An easy test would be to check whether `mid` is equal to `lo`. Having that value could indicate that the first recursive call has completed on the `[lo, mid)` range, while a higher value indicates that the higher subrange is done, and the last part of the processing is next.

The implementation of the `MergesortViaStack.py` module shown in [Listing 6-8](#) uses this approach of altering the `mid` value. It defines the `Mergesort` class in the same fashion as that of [Listing 6-4](#). Inside the constructor, it creates a linked-list stack to hold the problem descriptions to be processed. The overall problem description is pushed onto the `__todo` stack before calling the `mergesort()` method to sort the unordered array. That initial problem description is the list `[0, None, n]` to indicate that it should sort the whole range `[0, n)` of the array, and the `mid` variable is initially undefined.

Listing 6-8 *The MergesortViaStack.py Module*

```
def identity(x): return x           # Identity function

from Array import *
from LinkStack import *

class Mergesort(object):           # An object to mergesort Arrays
    def __init__(self,             # Constructor, takes the unordered
        unordered,                # array and orders its items by using
        key=identity):            # mergesort on their keys
        self.__arr = unordered    # Array starts unordered
        self.__key = key          # Key func. returns sort key of item
        n = len(unordered)        # Get number of items
        self.__work = Array(n)    # A work array of the same length
        for i in range(n):        # is needed to rearrange the items
            self.__work.insert(None) # Work array is filled with None
        self.__todo = LinkStack() # Stack to manage subproblems
        self.__todo.push([0, None, n]) # Add overall problem description
        self.mergesort()          # Call mergesort on problem

    def mergesort(self):           # Perform mergesort on subrange
        while not self.__todo.isEmpty(): # Loop until no problems remain
```

```

lo, mid, hi = self.__todo.peek() # Get [lo, mid, hi] values
print('Mergesort working on [lo, mid, hi] =', # Show progress
      self.__todo.peek(), 'at depth', len(self.__todo))
if lo + 1 >= hi: # If subrange has 1 or fewer items,
    self.__todo.pop() # then done, and remove problem
    if self.__todo.isEmpty(): # If that was 1st problem
        return # then everything is done
    self.__todo.peek()[1] = lo # Otherwise, store lo index in
                                # caller's problem description for
                                # 'mid' to signal completion
elif mid is None: # If mid is None, need to compute it
    mid = (lo + hi) // 2 # Find middle index, and add subtask
    self.__todo.push( # for the lower half of subrange
        [lo, None, mid])
elif (mid == lo): # If mid is lo, lower half is done
    self.__todo.push( # Add subtask for upper half of
        [(lo + hi) // 2, None, hi]) # subrange
else: # Both lower half and upper half done
    print('Merging ranges [' , lo, ', ', mid, '] with [' ,
          mid, ', ', hi, ']')
    self.merge(lo, mid, hi) # Merge the 2 sorted halves
    self.__todo.pop() # Remove completed problem
    if self.__todo.isEmpty(): # If that was the 1st problem,
        return # then everything is done
    self.__todo.peek()[1] = lo # Otherwise, signal caller
raise Exception('Empty stack in mergesort')

```

As with the `triangular_via_stack()` function, the `mergesort()` method is a loop that processes problem descriptions on the `__todo` stack until it is empty. It takes the topmost problem description and copies the fields into the same variable names used in the recursive version, `lo`, `mid`, and `hi`. After printing the description record, it goes to the first part of the recursive process, checking the base case. If it is a one-cell or shorter subrange, nothing needs sorting, so it pops off the problem description. If the stack is now empty, then completing this recursive call means everything is done and the stack-based procedure can return. Otherwise, it must signal that it has completed the recursive call by updating the `mid` variable of the caller's problem description. It sets the middle element of the caller's problem description (stored on the top of the stack) to the `lo` value of the subrange just processed. The "caller" had originally put `None` for that field, and it is now updated to an integer within the subrange the caller was processing.

Setting the `mid` value of the caller to the `lo` value of the problem just solved may seem as though it's going to cause an error in the caller due to that

unexpected change. Remember, however, that the stack-based approach is going to use that value to determine what step is being performed, and the caller can easily reconstruct `mid` from its `lo` and `hi` values, which do not change. Note also that by setting this field to `lo`, the caller will be able to tell if the first subrange or second subrange has just been processed.

The next step after handling the base case depends on the value of `mid`. If it's not set, then this is the first part of the processing and the range is bigger than 1 cell. It calculates what `mid` should be and pushes the lower half subrange on the `__todo` stack. Note that `mid` must be at least 1 larger than `lo` because the base case handled when `lo` and `hi` are separated by less than 2.

The next test looks to see if `mid` is the same as `lo`. That indicates the problem description on the top of the stack just finished the recursive call on the lower half, so it pushes a description of the upper half subrange onto the stack. Note that `mid` was set to `lo` by the first recursive call, so the method must recompute the higher subrange starting index as `(lo + hi) // 2`.

The final `else` clause handles the case after returning from the second recursive call, the one that sorted the upper half subrange. The value of `mid` is not `None` nor is it the same as `lo`, so it must have been set to something above `lo` by the recursive call. Now the two subranges can be merged using the exact same `merge()` method shown in [Listing 6-5](#). When they're merged, the problem description can be popped off the stack. If the stack is empty, everything is done. Otherwise, it updates the “caller” problem description on top of the stack to mark the beginning of the range that was just finished. If the “caller” shared the same value for `lo`, then this pass through the loop finished its lower half. If the “caller” value for `lo` is different (lower) than this pass's value, then this pass mergesorted the upper half of the `lo` to `hi` range.

The final line of the `mergesort()` method raises an exception if the stack is emptied unexpectedly. Normally, the return statements in the loop body handle the end of processing.

The stack-based implementation of `mergesort()` in [Listing 6-8](#) includes two `print` statements to show what's going on. Here's a partial transcript of the end of the execution on a 14-cell array.

```
Mergesort working on [lo, mid, hi] = [0, 0, 14] at depth 1
Mergesort working on [lo, mid, hi] = [7, None, 14] at depth 2
Mergesort working on [lo, mid, hi] = [7, None, 10] at depth 3
Mergesort working on [lo, mid, hi] = [7, None, 8] at depth 4
```

Mergesort working on [lo, mid, hi] = [7, 7, 10] at depth 3
Mergesort working on [lo, mid, hi] = [8, None, 10] at depth 4
Mergesort working on [lo, mid, hi] = [8, None, 9] at depth 5
Mergesort working on [lo, mid, hi] = [8, 8, 10] at depth 4
Mergesort working on [lo, mid, hi] = [9, None, 10] at depth 5
Mergesort working on [lo, mid, hi] = [8, 9, 10] at depth 4
Merging ranges [8 , 9) with [9 , 10)
Mergesort working on [lo, mid, hi] = [7, 8, 10] at depth 3
Merging ranges [7 , 8) with [8 , 10)
Mergesort working on [lo, mid, hi] = [7, 7, 14] at depth 2
Mergesort working on [lo, mid, hi] = [10, None, 14] at depth 3
Mergesort working on [lo, mid, hi] = [10, None, 12] at depth 4
Mergesort working on [lo, mid, hi] = [10, None, 11] at depth 5
Mergesort working on [lo, mid, hi] = [10, 10, 12] at depth 4
Mergesort working on [lo, mid, hi] = [11, None, 12] at depth 5
Mergesort working on [lo, mid, hi] = [10, 11, 12] at depth 4
Merging ranges [10 , 11) with [11 , 12)
Mergesort working on [lo, mid, hi] = [10, 10, 14] at depth 3
Mergesort working on [lo, mid, hi] = [12, None, 14] at depth 4
Mergesort working on [lo, mid, hi] = [12, None, 13] at depth 5
Mergesort working on [lo, mid, hi] = [12, 12, 14] at depth 4
Mergesort working on [lo, mid, hi] = [13, None, 14] at depth 5
Mergesort working on [lo, mid, hi] = [12, 13, 14] at depth 4
Merging ranges [12 , 13) with [13 , 14)
Mergesort working on [lo, mid, hi] = [10, 12, 14] at depth 3
Merging ranges [10 , 12) with [12 , 14)
Mergesort working on [lo, mid, hi] = [7, 10, 14] at depth 2
Merging ranges [7 , 10) with [10 , 14)
Mergesort working on [lo, mid, hi] = [0, 7, 14] at depth 1
Merging ranges [0 , 7) with [7 , 14)
After applying Mergesort, array contains 14 items

6
18
19
19
43
45
46
49
56
70
72
80
85
95

The transcript starts right after the lower half of the array has been sorted. The top of the stack is `[0, 0, 14]`. That indicates that it should be sorting the range `[0, 14]` and `mid` has been set to 0. That was done by processing the "recursive" subproblem for the lower half, which isn't shown but must have processed the `[0, 7)` range. It now pushes on the next subproblem to solve `[7, None, 14]`, the upper half of the array. Because the size of that subrange is larger than 1, it continues pushing on smaller subranges until it gets to `[7, None, 8]` at a stack depth of 4. Because that is a subrange of one cell, it is handled by the base case.

The single cell base case causes `mergesort()` to return to the problem at depth 3, but with the `mid` value of its caller now set to 7, so the full description is `[7, 7, 10]`. That is a three-cell subrange that must be recursively divided as before. After several more subdivisions to get to the base cases, the transcript has its first merges, `[8, 9)` with `[9, 10)`. Those are both single cell subranges. After being merged, the subrange `[8, 10)` is now sorted. The next merge combines `[7, 8)` with `[8, 10)`, to complete the three-cell subrange `[7, 10)`.

The rest of the transcript continues handling the various subranges and merging their results. The second-to-last merge merges `[7, 10)` with `[10, 14)`, the upper half of the entire array. The final merge combines the upper half with the lower half (whose transcript is not shown) to produce the fully sorted array.

The key point to remember is that you can prototype a new function, procedure, or method using recursion and later convert it to a stack-based or simple loop form later. The recursive approach is conceptually simpler and can be easier to write, whereas the stack or loop forms gain efficiency.

Some Interesting Recursive Applications

Let's look briefly at some other situations in which recursion is useful. You will see from the diversity of these examples that recursion can pop up in unexpected places. Here, we examine three problems: raising a number to a power, fitting items into a knapsack, and choosing members of a team. We explain the concepts and leave the implementations as exercises.

Raising a Number to a Power

Most scientific calculators, programming languages, and math libraries allow you to raise a number to an arbitrary power. They usually have a function like `power(x, y)` or a key labeled something like x^y or $x^{\wedge}y$. How would you do this calculation if your programming language didn't have this function or a math library? You might assume you would need to multiply x by itself y times. That is, if x was 2 and y was 8 (2^8), you would carry out the arithmetic for $2 \times 2 \times 2 \times 2 \times 2 \times 2 \times 2 \times 2$. For large values of y , however, this approach might prove longer than necessary. Is there a quicker way?

One solution is to rearrange the problem so that you multiply by multiples of 2 whenever possible, instead of by 2 itself. Take 2^8 as an example. Eventually, you must involve eight 2s in the multiplication process. Let's say you start with $2 \times 2 = 4$. You've used up two of the 2s, but there are still six to go. You now have a new number to work with: 4. So you try $4 \times 4 = 16$. This uses four 2s (because each 4 is two 2s multiplied together). You need to use up four more 2s, but now you have 16 to work with, and $16 \times 16 = 256$ uses exactly eight 2s (because each 16 used four 2s).

So, you've found the answer to 2^8 with only three multiplications instead of seven. That's $O(\log N)$ time instead $O(N)$.

Can you make this process into an algorithm that a computer can execute? The scheme is based on the mathematical equality $x^y = (x^2)^{y/2}$. In this example, $2^8 = (2^2)^{8/2}$, or $2^8 = (2^2)^4$. This is true because raising a power to another power is the same as multiplying the powers.

Remember the assumption, however, that the computer can't raise a number to a power, so it can't handle $(2^2)^4$ directly. Let's see if you can transform this into an expression that involves only multiplication. The trick is to start by substituting a new variable for 2^2 .

Let's say that $2^2 = a$. Then 2^8 equals $(2^2)^4$, which is a^4 . According to the original equality, however, a^4 can be written $(a^2)^2$, so $2^8 = (a^2)^2$.

Again, you can substitute a new variable for a^2 , say $a^2 = c$, then $(c)^2$ can be written $(c^2)^1$, which also equals 2^8 , if you apply all the substitutions.

Now you have a problem you can handle with simple multiplication: c times c .

You can imbed this scheme in a recursive method—let's call it `power()`—for calculating powers. The arguments are b and p , and the method returns b^p . You

don't need to worry about variables like a and c anymore because b and p get new values each time the method calls itself. The recursive call arguments are $b \times b$ and $p/2$. For $b=2$ and $p=8$, the sequence of arguments and return values would be

```
b=2, p=8
b=4, p=4
b=16, p=2
b=256, p=1
Returning 256, b=256, p=1
Returning 256, b=16, p=2
Returning 256, b=4, p=4
Returning 256, b=2, p=8
```

When p is 1, you return b . The answer, 256, is passed unchanged back up the sequence of methods.

We've shown an example in which p is an even number throughout the entire sequence of divisions. This will not always be the case. Here's how to revise the algorithm to deal with the situation where p is odd. Use integer division on the way down and don't worry about a remainder when dividing p by 2. However, during the return process, whenever p is an odd number, do an additional multiplication by b . Here's the sequence for 3^{18} :

```
b=3, p=18
b=9, p=9
b=81, p=4
b=6561, p=2
b=43046721, p=1
Returning 43046721, b=43046721, p=1
Returning 43046721, b=6561, p=2
Returning 43046721, b=81, p=4
Returning 387420489, b=9, p=9 # p is odd; so multiply by b
Returning 387420489, b=3, p=18
```

The Knapsack Problem

The knapsack problem is a classic in computer science. In its simplest form, it involves trying to fit items of different weights into a knapsack so that the knapsack ends up with a specified total weight. You don't need to fit in all the items. It's equivalent to the problem of coming up with exact change for a particular value given a specific set of coins.

For example, suppose you want the knapsack to weigh exactly 20 kilograms, and you have five items, with weights of 11, 8, 7, 6, and 5 kilograms. For small numbers of items, humans are pretty good at solving this problem by inspection. So, you can probably figure out that only the 8, 7, and 5 combination of items adds up to 20.

If you want a computer to solve this problem, you need to give it more detailed instructions. Here's the algorithm:

1. If at any point in this process the sum of the selected items adds up to the target, you're done.
2. Start by selecting the first item. The remaining items must add up to the knapsack's target weight minus the first item; this is a new target weight.
3. Try, one by one, each of the possible combinations of the remaining items. Notice, however, that it doesn't really need to try all the combinations, because whenever the sum of the items is more than the target weight, it can stop adding items.
4. If none of the combinations work, discard the first item, and start the whole process again with the second item.
5. Continue this process with the third item and so on until all the combinations have been tried, at which point it knows there is no solution.

In the preceding example, the algorithm starts with the 11 kg weight. Now you want the remaining items to add up to 9 (20 minus 11). Of these, you start with 8, which is too small. Now you want the remaining items to add up to 1 (9 minus 8). You start with 7, but that's bigger than 1, so you try 6 and then 5, which are also too big. You've run out of items, so you know that any combination that includes 8 won't add up to 9. Next, you try 7, so now you're looking for a target of 2 (9 minus 7). You continue in the same way, as summarized here:

```
Items: 11, 8, 7, 6, 5
=====
11          // Target = 20, 11 is too small
11, 8       // Target = 9, 8 is too small
11, 8, 7    // Target = 1, 7 is too big
11, 8, 6    // Target = 1, 6 is too big
11, 8, 5    // Target = 1, 5 is too big. No more items
11, 7       // Target = 9, 7 is too small
```

```

11, 7, 6    // Target = 2, 6 is too big
11, 7, 5    // Target = 2, 5 is too big. No more items
11, 6       // Target = 9, 6 is too small
11, 6, 5    // Target = 3, 5 is too big. No more items
11, 5       // Target = 9, 5 is too small. No more items
8,          // Target = 20, 8 is too small
8, 7        // Target = 12, 7 is too small
8, 7, 6     // Target = 5, 6 is too big
8, 7, 5     // Target = 5, 5 is just right. Success!

```

As you may recognize, a recursive routine can pick the first item, and, if the item is smaller than the target, the routine can call itself with a new target to investigate the sums of all the remaining items.

Combinations: Picking a Team

In mathematics, a **combination** is a selection of things in which their order doesn't matter. For example, suppose there is a group of five candidate astronauts from which you want to select a team of three to go on a multiyear journey through the solar system. There is concern, however, about how the team members will get along and handle all the important tasks, so you decide to list all the possible teams; that is, all the possible combinations of three people. You can name the candidates A, B, C, D, and E. You want a program that would show the 10 possible combinations:

ABC, ABD, ABE, ACD, ACE, ADE, BCD, BCE, BDE, CDE

How would you write such a program? It turns out there's an elegant recursive solution. It involves dividing these combinations into two groups: those that begin with A and those that don't. Suppose you abbreviate the idea of 3 people selected from a group of 5 as $C(5, 3)$. The $C()$ function produces a set of teams. Let's say n is the size of the group of candidates and k is the size of a team, that is $C(n, k)$. A theorem says that if A is the first of the n candidates, then

$$C(n, k) = A \otimes C(n - 1, k - 1) + C(n - 1, k)$$

where $A \otimes C(e)$ means add (cross) A to all the combinations produced by $C(e)$ and $C(n - 1, j)$ means find all combinations of size j from the candidates other than A, which is one of the n candidates. For the example of 3 people selected from a group of 5, you have

$$C(5, 3) = A \otimes C(4, 2) + C(4, 3)$$

This example breaks a large problem into two smaller ones. Instead of selecting from a group of 5, you're selecting twice from a group of 4, the candidates other than A. First, all the ways to select 2 people from 4 candidates, then all the ways to select 3 people from 4 candidates.

There are 6 ways to select 2 people from a group of 4. In the $C(4, 2)$ term—which you can call the left term—these 6 combinations are

BC, BD, BE, CD, CE, DE

The theorem tells you to add (cross) A to all of these 2-person combinations:

ABC, ABD, ABE, ACD, ACE, ADE

There are four ways to select 3 people from a group of 4. In the $C(4, 3)$ term—the right term—you have

BCD, BCE, BDE, CDE

When these 4 combinations from the right term are added to the 6 from the left term, you get the 10 combinations for $C(5, 3)$.

You can apply the same decomposition process to each of the groups of 4. For example, $C(4, 2)$ is $B \times C(3, 1)$ added to $C(3, 2)$, where both A and B have been taken out of the candidates. As you can see, this is a natural place to apply recursion.

You can think of this problem visually with $C(5, 3)$ as a single node on the top row, $C(4, 2)$ and $C(4, 3)$ as nodes on the next row, and so on, where the nodes correspond to recursive function calls. [Figure 6-19](#) shows what this looks like for the $C(5, 3)$ example.



Figure 6-19 *Picking a team of 3 from a group of 5*

The base cases are combinations that make no sense: those with a 0 for the team size and those where the team size is greater than the number of candidates. The combination $C(1, 1)$ is valid, but there's no point trying to break it down further. In the figure, grayed-out boxes show the base cases where recursion ends.

The recursion depth corresponds to the candidates: the node on the top row represents candidate A with its two choices of whether to include A or not in the team. The two nodes on the next row represent candidate B, and so on. If there are 5 candidates, you'll have 5 levels.

As the algorithm descends the rows, it needs to remember the sequence of candidates visited. Here's how to do that: Whenever it makes a call to a left term, it records the node it's leaving by adding its letter to a sequence. These left calls and the letters to add to the sequence are shown by the darker and thicker lines in the figure. The sequence needs to be passed back up on the returns.

To record all the combinations, the algorithm can add them to a list or display them as they are found. After a complete team is assembled, it's either added to the list or printed (or both).

Summary

- A recursive method calls itself repeatedly, with different argument values each time.
- Some value(s) of its arguments causes a recursive method to return without calling itself. This is called a base case.
- When the innermost instance of a recursive method returns, the process "unwinds" by completing pending instances of the method, going from the latest back to the original call.
- The n th triangular number is the sum of all the numbers from 1 to n . (*Number* means *integer* in this context.) For example, the triangular number of 4 is 10, because $4+3+2+1 = 10$.

- The factorial of an integer number is the product of itself and all numbers smaller than itself. For example, the factorial of 4 is $4 \times 3 \times 2 \times 1 = 24$.
- Both triangular numbers and factorials can be calculated using either a recursive method or a simple loop.
- The anagrams of a word (all possible permutations of its N letters) can be found recursively by placing the leftmost letter at every position within all the anagrams of the rightmost $N-1$ letters.
- A binary search can be carried out recursively by checking which half of a sorted range the search key is in, and then doing the same kind of search within that half.
- The Tower of Hanoi puzzle consists of three spindles and an arbitrary number of disks stacked in a tower on one of the spindles.
- Solving the Tower of Hanoi involves moving disks one at a time between spindles until they are all stacked on the goal spindle, without ever placing a larger disk on top of a smaller disk.
- The Tower of Hanoi puzzle can be solved recursively by moving all but the bottom disk of a pyramid to an intermediate tower, moving the bottom disk to the destination tower, and finally moving the remaining pyramid to the destination.
- The number of steps needed to solve a Tower of Hanoi puzzle with N disks is $O(2^N)$.
- $O(2^N)$ is significantly worse than $O(N^2)$ complexity.
- Merging two sorted arrays means creating a third array that contains all the elements from both arrays in sorted order.
- In mergesort, one-element subarrays of a larger array are merged into two-element subarrays, two-element subarrays are merged into four-element subarrays, and so on until the entire array is sorted.
- mergesort requires $O(N \times \log N)$ time for both comparisons and copies.
- mergesort requires a workspace equal in size to the original array.

- For triangular numbers, factorials, anagrams, and the binary search, the recursive method contains only one call to itself. (There are two shown in the code for the binary search, but only one is used on any given pass through the method's code.)
- For the Tower of Hanoi and mergesort, the recursive method contains two calls to itself.
- Any operation that can be carried out with recursion can be carried out with a stack.
- There is a process to convert recursive functions and procedures into stack-based algorithms.
- A recursive approach may be inefficient. If so, it can sometimes be replaced with a simple loop, a stack-based approach, and very occasionally without iteration.
- Prototyping in recursive form is simpler conceptually and can be useful to test an algorithm's accuracy and correctness.

Questions

These questions are intended as a self-test for readers. Answers may be found in [Appendix C](#).

1. Which of the following is **not** a characteristic of recursive routines?
 - a. They call themselves.
 - b. Each call performs its work on a smaller version of the same problem.
 - c. When a smaller version of the problem is too complex, control passes back to the caller to try a different approach.
 - d. Some versions of the problem don't require calling the recursive routine.
2. If a program calls `triangular(100)` using the definition in [Listing 6-1](#), what is the maximum number of "copies" of the `triangular()` function in execution that exist at any one time?
3. Where are the copies of the argument passed to the `triangular()` function, mentioned in question 2, stored?

- a. in a variable in the `triangular()` function
 - b. in a field of the `Triangular` class
 - c. in a placeholder variable of the problem description record
 - d. on a stack
4. Assume a call to `triangular(100)` as in question 2. What is the value of `nth` right after the `triangular()` function first returns a nonbase case value?
5. True or False: In the `triangular()` function of [Listing 6-1](#), the return values are stored on the stack.
6. In the `anagrams()` function, at a certain depth of recursion, assume a version of the function is working with the string "que". When this method calls a new version of itself, what letters will the new version be working with?
7. In the section "A Recursive Binary Search," the original, loop-based form was compared with a recursive form of the `find()` method. Which of the following is **not** true?
- a. The search range starts with the whole array, and only the recursive version can work on a subrange passed through arguments.
 - b. Both forms of the program divide the search range repeatedly in half.
 - c. If the key is not found, the loop version returns when the range bounds cross, but the recursive version finishes when the recursive depth is more than half the initial search range.
 - d. If the key is found, the loop version returns from the entire method, whereas the recursive version returns from one level of recursion.
8. What kind of subproblem is solved in the recursive calls of the `TowerOfHanoi.solve()` method ([Listing 6-3](#)) as compared to the overall problem?
9. The algorithm in the `TowerOfHanoi.solve()` method involves
- a. dividing the number of disks on the source spindle in half.
 - b. changing which spindles are the source and destination.
 - c. removing the small disks from all the spindles to move the large disks.

- d. moving one small disk and then a stack of larger disks.
- 10. Which is **not** true about the `Mergesort.merge()` method in Listing 6-5?
 - a. Its algorithm can handle arrays of different sizes.
 - b. It must search the target array to find where to put the next item.
 - c. It is not recursive.
 - d. It continuously copies the smallest item irrespective of what input array it's in.
- 11. The disadvantage of mergesort is that
 - a. it is not recursive.
 - b. it uses more memory.
 - c. although faster than the insertion sort, it is much slower than bubble sort.
 - d. it is complicated to implement.
- 12. Using the recursive version of `mergesort()` in Listing 6-4, what recursive depth will be reached in a call to sort an array of 1,024 cells?
- 13. In addition to a loop, a _____ can often be used instead of recursion.
- 14. In the procedure outlined for converting a recursive function to a nonrecursive algorithm, how are function arguments and local variables of the recursive version stored?
- 15. In the procedure outlined for converting a recursive function to a nonrecursive function, what test is used to decide when the function should return?

Experiments

Carrying out these experiments will help to provide insights into the topics covered in the chapter. No programming is involved.

- 6-A Use a Python interpreter to load the definitions for `triangular_loop()` and `triangular()` shown in the "Triangular Numbers" section. Run both routines to get the results for the 100th, 1,000th, and 10,000th

triangular number. What's different about the results? Does one routine take noticeably longer?

- 6-B This chapter discussed triangular numbers and factorials. It showed recursive algorithms to generate them and a recursive algorithm to generate anagrams. How are anagrams related to one or both of those numeric sequences?
- 6-C The mergesort algorithm is another sorting algorithm like the ones introduced in [Chapter 3](#). Do you think it is stable? In other words, will any two items in the input array with equal keys remain in the same relative ordering in the resulting array? Why or why not? Thinking about the simple base cases first can be helpful. The recursive nature of the algorithm determines the rest.
- 6-D In [Chapter 3](#), you saw that in the best case, an insertion sort would take $O(N)$ time. Can you think of what the best case input array would be for mergesort? Would sorting that best case array take less than its worst case time, $O(N \times \log N)$?

Programming Projects

Writing programs to solve the Programming Projects helps to solidify your understanding of the material and demonstrates how the chapter's concepts are applied. (As noted in the Introduction, qualified instructors may obtain completed solutions to the Programming Projects on the publisher's website.)

- 6.1 Suppose you are asked to write a program on an embedded processor that has very limited processing capability. This particular processor's Python interpreter can only do addition and subtraction, not multiplication, and that's needed for this project. You program your way out of this quandary by writing a recursive function, `mult()`, that performs multiplication of x and y by adding x to itself y times. Its arguments are integers, and its return value is the product of x and y . Write such a method and a test program to call it. Does the addition take place when the function calls itself or when it returns?
- 6.2 Every positive integer can be divided into a set of integer factors. The minimum set of factors must be a collection of prime numbers, where a prime number is one that is only evenly divisible by 1 and itself. Write a recursive function, `factor()`, that returns the list of integer factors of x .

If you haven't worked with factoring before, it is helpful to know that only the factors between 2 and the square root of x need to be tested. Instead of doing those with a loop, add a second, optional parameter to `factor(x, lowest)` that is the lowest possible integer factor of x . The recursive function should check for the base case(s), and make recursive calls based on whether `lowest` evenly divides x (`x % lowest == 0`). If it does, then add `lowest` to the factors of x divided by `lowest`. If `lowest` doesn't evenly divide x , then look for factors with the next higher possible factor. Ideally, the next higher factor would always be the next higher prime number. There are algorithms for that, but for this exercise, it's sufficient to try the next higher integer. You may use Python's built-in `list` structure to assemble the factors or the `LinkedList` structure of [Chapter 5](#). Note that a factor can appear more than once in the final list. For extra utility, handle requests for the factors of negative integers by returning the factors of the positive version but with the factor 1 replaced by -1 . Test your function on some compound (nonprime) and prime numbers and the special cases of 0 and 1.

- 6.3 Implement the recursive approach to raising a number to a power, as described in the "Raising a Number to a Power" section near the end of this chapter. Write the recursive `power()` function. For extra utility, make use of this transformation to handle negative integer exponents:

$$x^y = \frac{1}{x^{-y}} \quad \text{when } y < 0$$

Test your function on several combinations including positive and negative integers and the special cases where the exponent is 0 and 1 ($x^0 = 1$ and $x^1 = x$).

- 6.4 Write a program that solves the knapsack problem for an arbitrary knapsack capacity and series of weights. Assume the weights are stored in an array. Hint: The arguments to the recursive `knapsack()` function are the target weight, the array or weights, and the index into that array where the remaining items start. The `knapsack()` function should either return a list of weights or print each list of weights that add up to the target weight. You may use Python's built-in `list` structure to assemble the lists or the `LinkedList` structure of [Chapter 5](#).

6.5 Implement a recursive approach to showing all the teams of size k that can be created from a group of n people. For this exercise, represent the n people with a single character, for example, A, B, T, Z. The list of all possible candidates can be represented as a string, for example, 'ABTZ'. The recursive `teams()` function should take the candidates string and k as parameters and generate a list of strings that are the different teams. Teams are formed by concatenating character strings.

7. Advanced Sorting

In This Chapter

- [Shellsort](#)
- [Partitioning](#)
- [Quicksort](#)
- [Radix Sort](#)
- [Timsort](#)

We started discussing sorting in the aptly titled [Chapter 3](#), “[Simple Sorting](#).” The sorts described there—the bubble, selection, and insertion sorts—are easy to implement but rather slow. In [Chapter 6](#), “[Recursion](#),” we described the mergesort. It runs much faster than the simple sorts but requires twice as much space as the original array; this is often a serious drawback.

This chapter covers two advanced approaches to sorting in detail: Shellsort and quicksort. These sorts both operate much faster than the simple sorts: the Shellsort in about $O(N \times (\log N)^2)$ time, and quicksort in $O(N \times \log N)$ time. Neither of these sorts requires a large amount of extra space, as mergesort does. The Shellsort is almost as easy to implement as mergesort although quicksort is the fastest of all the general-purpose sorts that don’t require a lot of memory.

We examine the Shellsort first. Quicksort is based on the idea of partitioning, so we examine that separately, before examining quicksort itself. We conclude the chapter with short descriptions of radix sort and Timsort, which are quite different approaches to fast sorting.

Shellsort

The Shellsort is named for Donald L. Shell, the computer scientist who discovered it in 1959. It's based on the insertion sort but adds a new feature that dramatically improves the insertion sort's performance.

The Shellsort is good for medium-sized arrays, depending on the particular implementation. It's not quite as fast as quicksort and other $O(N \times \log N)$ sorts, so it's not optimum for very large collections. Shellsort, however, is much faster than the $O(N^2)$ sorts like the selection sort and the insertion sort, and it's very easy to implement. The code is short and simple.

Shellsort's worst-case performance is not significantly worse than its average performance. (Later in this chapter we look into that the worst-case performance for quicksort, which can be much worse unless precautions are taken.) Some experts (see Sedgewick in [Appendix B](#), “[Further Reading](#)”) recommend starting with a Shellsort for almost any sorting project and changing to a more advanced sort, like quicksort, only if Shellsort proves too slow in practice.

Insertion Sort: Too Many Copies

Because Shellsort is based on the insertion sort, you might want to review the “[Insertion Sort](#)” section in [Chapter 3](#). Recall that partway through the insertion sort the items to the left of a marker are partially sorted (sorted among themselves), and the items to the right are not. The algorithm removes the item at the marker and stores it in a temporary variable. Then, beginning with the item to the left of the newly vacated cell, it shifts the sorted items right one cell at a time, until the item in the temporary variable can be reinserted in sorted order.

Here's the problem with the insertion sort. Suppose a small item is on the far right, where the large items should be. To move this small item to its proper place on the left, all the intervening items (between the place where it is and where it should be) must be shifted one space right. This step takes close to N copies, just for one item. Not all the items must be moved a full N spaces, but the average item must be moved $N/2$ spaces, which mean the full process takes N times $N/2$ shifts for a total of $N^2/2$ copies. Thus, the performance of insertion sort is $O(N^2)$.

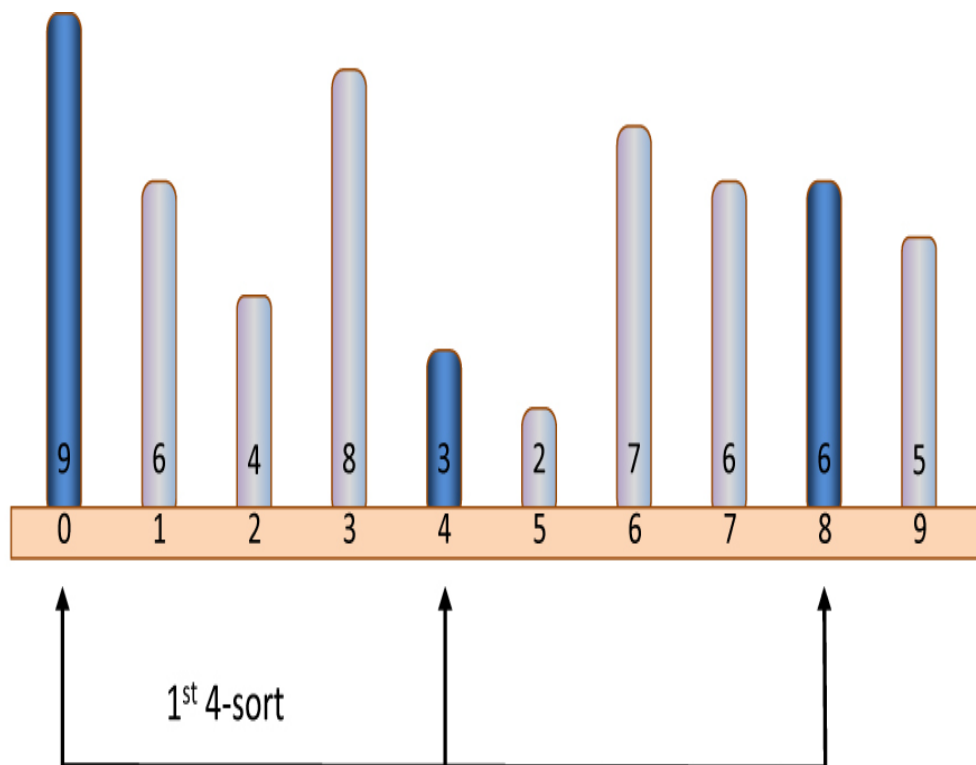
This performance could be improved if you could somehow move a smaller item many spaces to the left without shifting all the intermediate items

individually.

N-Sorting

The Shellsort achieves these large shifts by insertion-sorting widely spaced elements. After they are sorted, Shellsort sorts somewhat less widely spaced elements, and so on. The spacing between elements for these sorts is called the *increment* and is traditionally represented by the letter h . [Figure 7-1](#) shows the first step in the process of sorting a 10-element peg board (array) with an increment of 4. Here the elements at indices 0, 4, and 8 are sorted.

Unsorted



After 1st 4-sort

